DATA 612 · Statistical Programming in R · Week 1

1.3 Working with Files and Paths

Find a file, tell R where it is, import it, and save the result.

When working with data in R, we often need to:

1. Find a file on the computer.
2. Tell R where the file is located.
3. Understand how the data inside the file is organized.
4. Choose the appropriate function to import it.
5. Inspect and work with the data.
6. Save or export the results.

A useful workflow is:

```
Find → Path → Identify Format → Import → Inspect → Work → Export
```

Work in an RStudio project so `getwd()` is the project folder. Download students.csv and place it in a `data/` folder inside that project (or in the project folder itself). Note **1.2** is how you install and load the tidyverse. The `read_csv()` examples below assume `library(tidyverse)` has already been run, or you can write `readr::read_csv()`.

---

1. Files, Folders, and Paths

A **path** tells R where a file or folder is located.

Suppose our project has the following structure:

```
my_project/
│
├── analysis.R
│
├── data/
│   └── students.csv
│
└── output/
```

The file `students.csv` is located inside the `data` folder.

There are two main ways to describe its location:

- Absolute path
- Relative path

2. Absolute Paths

An **absolute path** gives the complete location of a file on the computer.

For example:

```
"C:/Users/John/Documents/my_project/data/students.csv"
```

We could therefore write:

```
students <- read.csv(
  "C:/Users/John/Documents/my_project/data/students.csv"
)
```

This may work on John's computer, but probably not on someone else's computer.

For example, another student may have the project at:

```
C:/Users/Sara/Desktop/my_project/
```

Therefore, absolute paths can make code difficult to share. Do not put them in work you submit. Use `/` in paths in R, even on Windows.

---

3. The Working Directory

R has a **working directory**.

Think of the working directory as:

> **R's current location or starting point.**

To find the current working directory:

```
getwd()
```

Think:

```
getwd() = Where am I?
```

For example, suppose:

```
getwd()
```

returns:

```
"C:/Users/John/Documents/my_project"
```

Then R currently considers `my_project` its starting location.

In RStudio, **File** → **New Project** pins the working directory to the project folder.

---

4. Relative Paths

A **relative path** describes the location of a file relative to our current location.

Suppose:

```
my_project/
│
├── analysis.R
│
├── data/
│   └── students.csv
│
└── output/
```

and our working directory is:

```
my_project/
```

Instead of writing:

```
"C:/Users/John/Documents/my_project/data/students.csv"
```

we can simply write:

```
"data/students.csv"
```

Therefore:

```
students <- read.csv("data/students.csv")
```

means:

> Starting from the current directory, go into the `data` folder and find `students.csv`.

Relative paths are usually preferable because they make projects easier to move and share.

---

5. Understanding `.`, `..`, and `/`

Three symbols are particularly useful when working with relative paths.

Symbol	Meaning
`.`	Current directory
`..`	Parent directory — one folder up
`/`	Separates folders

`.` means "here"

These are generally equivalent:

```
read.csv("data/students.csv")
```

and:

```
read.csv("./data/students.csv")
```

The second can be read as:

> Start here → enter `data` → find `students.csv`.

`..` means "go up one folder"

Suppose:

```
my_project/
│
├── data/
│   └── students.csv
│
└── scripts/
    └── analysis.R
```

If our current location is:

```
my_project/scripts/
```

then the relative path to `students.csv` is:

```
"../data/students.csv"
```

Read this as:

```
..             → go up to my_project
data/          → enter the data folder
students.csv   → find the file
```

So:

```
students <- read.csv("../data/students.csv")
```

Going up multiple folders

Suppose:

```
my_project/
│
├── data/
│   └── students.csv
│
└── R/
    └── week1/
        └── analysis.R
```

If our current directory is:

```
my_project/R/week1/
```

then:

```
"../../data/students.csv"
```

means:

```
..       → R/
../..    → my_project/
data/    → my_project/data/
```

---

6. Why Prefer Relative Paths?

Compare:

```
# Absolute path
read.csv(
  "C:/Users/John/Documents/my_project/data/students.csv"
)
```

with:

```
# Relative path
read.csv("data/students.csv")
```

If we send the entire `my_project` folder to another person, the absolute path will probably not work.

The relative path can still work because the relationship between the folders has not changed.

Therefore:

> **When working within an R project, use relative paths whenever possible.**

---

7. Finding Files in R

Before importing a file, we may need to determine where it is.

Start with:

```
getwd()
```

> Where am I?

Then:

```
list.files()
```

> What files and folders are here?

For example:

```
list.files()
```

might return:

```
"analysis.R" "data" "output"
```

We can look inside a particular folder:

```
list.files("data")
```

which might return:

```
"students.csv"
```

The function:

```
dir()
```

can also be used to list files.

---

8. Searching for a Particular Type of File

Suppose we want to find CSV files:

```
list.files(pattern = "\\.csv$")
```

To search inside subfolders as well:

```
list.files(
  recursive = TRUE,
  pattern = "\\.csv$"
)
```

This might return:

```
"data/students.csv"
```

To return full paths:

```
list.files(
  recursive = TRUE,
  pattern = "\\.csv$",
  full.names = TRUE
)
```

---

9. Does the File Exist?

Before importing:

```
file.exists("data/students.csv")
```

If R returns:

```
TRUE
```

the file exists at that path.

If R returns:

```
FALSE
```

check:

```
getwd()
list.files()
list.files("data")
```

For directories:

```
dir.exists("data")
```

---

10. Constructing Paths

R provides `file.path()`:

```
file.path("data", "students.csv")
```

For longer paths:

```
file.path("data", "raw", "students.csv")
```

This is useful for constructing paths from individual folder and file names.

---

11. Selecting a File Manually

Base R provides:

```
file.choose()
```

This opens a file-selection window.

For example:

```
path <- file.choose()
```

Then:

```
path
```

shows the selected path.

We could import the selected file using:

```
students <- read.csv(path)
```

This is convenient for quick interactive work.

For reproducible scripts, however, relative paths are usually preferable:

```
students <- read.csv("data/students.csv")
```

---

12. Before Importing: What Is Inside the File?

Finding the file is only part of the problem.

We also need to understand **how the data inside the file is organized**.

Many data files are **delimited text files**.

A **delimiter** is the character used to separate values.

For example:

```
name,age,grade
John,20,85
Sara,21,92
```

The values are separated by:

```
,
```

Therefore, this is **comma-separated data**.

Now consider:

```
name    age    grade
John    20     85
Sara    21     92
```

If the spaces between values are tab characters, this is **tab-separated data**.

Other delimiters are also possible:

```
John;20;85
```

uses a semicolon.

And:

```
John|20|85
```

uses a pipe character.

The course file `students.csv` uses commas. Its columns are `name`, `grade`, and `passed`. The `name,age,grade` lines above are only to show what a delimiter looks like.

---

13. How Do We Determine the Delimiter?

The file extension gives us a useful clue:

Extension	Usually means
`.csv`	Comma-separated values
`.tsv`	Tab-separated values
`.txt`	General text file; delimiter may vary

However:

> **The file extension is a clue, not a guarantee.**

Someone could name a semicolon-separated file `students.csv`.

Therefore, for an unfamiliar file, we can inspect its first few lines before importing it.

Base R provides:

```
readLines()
```

For example:

```
readLines(
  "data/students.csv",
  n = 3
)
```

R might display:

```
"name,age,grade"
"John,20,85"
"Sara,21,92"
```

We can clearly see commas.

Or R might display:

```
"name\tage\tgrade"
"John\t20\t85"
"Sara\t21\t92"
```

Here:

```
\t
```

represents a **tab character**.

Therefore, the data is tab-separated.

---

14. Choosing the Correct Import Function

Once we know the delimiter, we can choose an appropriate function.

Delimiter	Example	Base R	readr/tidyverse
Comma `,`	`John,20,85`	`read.csv()`	`read_csv()`
Tab `\t`	`John 20 85`	`read.delim()`	`read_tsv()`
Other	`John;20;85`	`read.table(sep = ";")`	`read_delim(delim = ";")`

For comma-separated data:

```
students <- read.csv("data/students.csv")
```

For tab-separated data:

```
students <- read.delim("data/students.tsv")
```

For another delimiter, such as `;`:

```
students <- read.table(
  "data/students.txt",
  sep = ";",
  header = TRUE
)
```

With `readr`:

```
students <- read_delim(
  "data/students.txt",
  delim = ";"
)
```

---

15. The Relationship Between the Base R Functions

The functions:

```
read.csv()
read.delim()
```

are convenient variations of the more general:

```
read.table()
```

The main difference is that they provide useful default arguments for particular file formats.

Conceptually:

```
read.table()
     │
     ├── comma separator → read.csv()
     │
     └── tab separator   → read.delim()
```

Therefore, these are not completely unrelated functions. The difference is **default arguments**, which note **1.4** discusses in more detail.

Similarly, `readr` provides:

```
read_csv()     → comma-separated

read_tsv()     → tab-separated

read_delim()   → specify another delimiter
```

---

16. Importing a CSV File

CSV stands for:

> **Comma-Separated Values**

For example:

```
name,age,grade
John,20,85
Sara,21,92
David,19,78
```

Each row represents an observation, and each column represents a variable.

Using Base R:

```
students <- read.csv("data/students.csv")
```

Using `readr`:

```
students <- read_csv("data/students.csv")
```

---

17. File vs. R Object

It is important to understand what happens when we import data:

```
FILE ON COMPUTER                OBJECT IN R

students.csv   --read.csv()-->   students
```

`students.csv` is a **file stored on the computer**.

`students` is an **R object stored in memory**.

They are not the same thing.

---

18. Inspecting Imported Data

Never assume that the file was imported correctly.

Start with:

```
head(students)
```

Then:

```
str(students)
```

and:

```
summary(students)
```

Other useful functions include:

```
tail(students)

dim(students)

nrow(students)
ncol(students)

names(students)
```

A useful habit is:

```
head(students)
str(students)
summary(students)
```

---

19. A Common Sign of the Wrong Delimiter

Suppose the original file contains:

```
name,age,grade
John,20,85
Sara,21,92
```

We expect **three columns**:

```
name | age | grade
```

If R imports everything as **one column**, something is probably wrong.

One common reason is:

> **The wrong delimiter was used.**

Therefore, if an imported dataset does not look correct:

```
head(students)
str(students)
```

Then inspect the original file:

```
readLines(
  "data/students.csv",
  n = 3
)
```

and verify the delimiter.

---

20. Working with Imported Data

Once imported, we normally work with the **R object**, not directly with the original file.

For example:

```
students$grade
```

Calculate:

```
mean(students$grade)
```

or create a variable:

```
students$passed <- students$grade >= 80
```

The R object has changed.

However:

> **Changing an R object does not automatically change the original file.**

The original:

```
data/students.csv
```

remains unchanged.

---

21. Exporting Data with Base R

To save our modified data:

```
write.csv(
  students,
  "output/students_updated.csv",
  row.names = FALSE
)
```

The workflow is:

```
students.csv
     │
     │ read.csv()
     ▼
R object: students
     │
     │ analyze / modify
     ▼
Modified R object
     │
     │ write.csv()
     ▼
students_updated.csv
```

---

22. Exporting with readr/tidyverse

Using `readr`:

```
write_csv(
  students,
  "output/students_updated.csv"
)
```

The main functions can therefore be summarized as:

Task	Base R	readr/tidyverse
Read comma-separated	`read.csv()`	`read_csv()`
Read tab-separated	`read.delim()`	`read_tsv()`
Read other delimiter	`read.table()`	`read_delim()`
Write CSV	`write.csv()`	`write_csv()`

---

23. Other Useful File Functions

Current directory

```
getwd()
```

List files

```
list.files()
```

Preview a text file

```
readLines("data/students.csv", n = 3)
```

Check whether a file exists

```
file.exists("data/students.csv")
```

Check whether a directory exists

```
dir.exists("data")
```

Construct a path

```
file.path("data", "students.csv")
```

Select a file manually

```
file.choose()
```

Get file information

```
file.info("data/students.csv")
```

Create a directory

```
dir.create("results")
```

Copy a file

```
file.copy(
  "data/students.csv",
  "output/students_copy.csv"
)
```

Rename a file

```
file.rename(
  "output/students_copy.csv",
  "output/backup.csv"
)
```

Delete a file

```
file.remove("output/backup.csv")
```

You do not need to memorize all these functions. The important goal is to understand what each type of operation does.

---

24. Other Common File Types

CSV is only one format.

File type	Base R / Common function	readr/tidyverse
`.csv`	`read.csv()`	`read_csv()`
`.tsv`	`read.delim()`	`read_tsv()`
`.txt`	`read.table()`	`read_delim()`
`.rds`	`readRDS()`	—
`.RData`	`load()`	—
`.xlsx`	`readxl::read_excel()`	`readxl::read_excel()`

The exact function changes depending on the file format, but the general workflow remains the same.

---

25. Saving R Objects

Sometimes we want to save an R object in R's own format rather than exporting it as CSV.

```
saveRDS(
  students,
  "output/students.rds"
)
```

Later:

```
students <- readRDS(
  "output/students.rds"
)
```

This preserves an R object rather than converting it to a general text format such as CSV.

---

26. Complete Base R Workflow

```
# 1. Where am I?
getwd()

# 2. What files are here?
list.files()
list.files("data")

# 3. Does the file exist?
file.exists("data/students.csv")

# 4. Look inside the file
readLines(
  "data/students.csv",
  n = 3
)

# 5. Import using the appropriate function
students <- read.csv(
  "data/students.csv"
)

# 6. Inspect
head(students)
str(students)
summary(students)

# 7. Work with the data
students$passed <- students$grade >= 80

# 8. Export
write.csv(
  students,
  "output/students_updated.csv",
  row.names = FALSE
)
```

27. The Same Basic Workflow with Tidyverse Tools

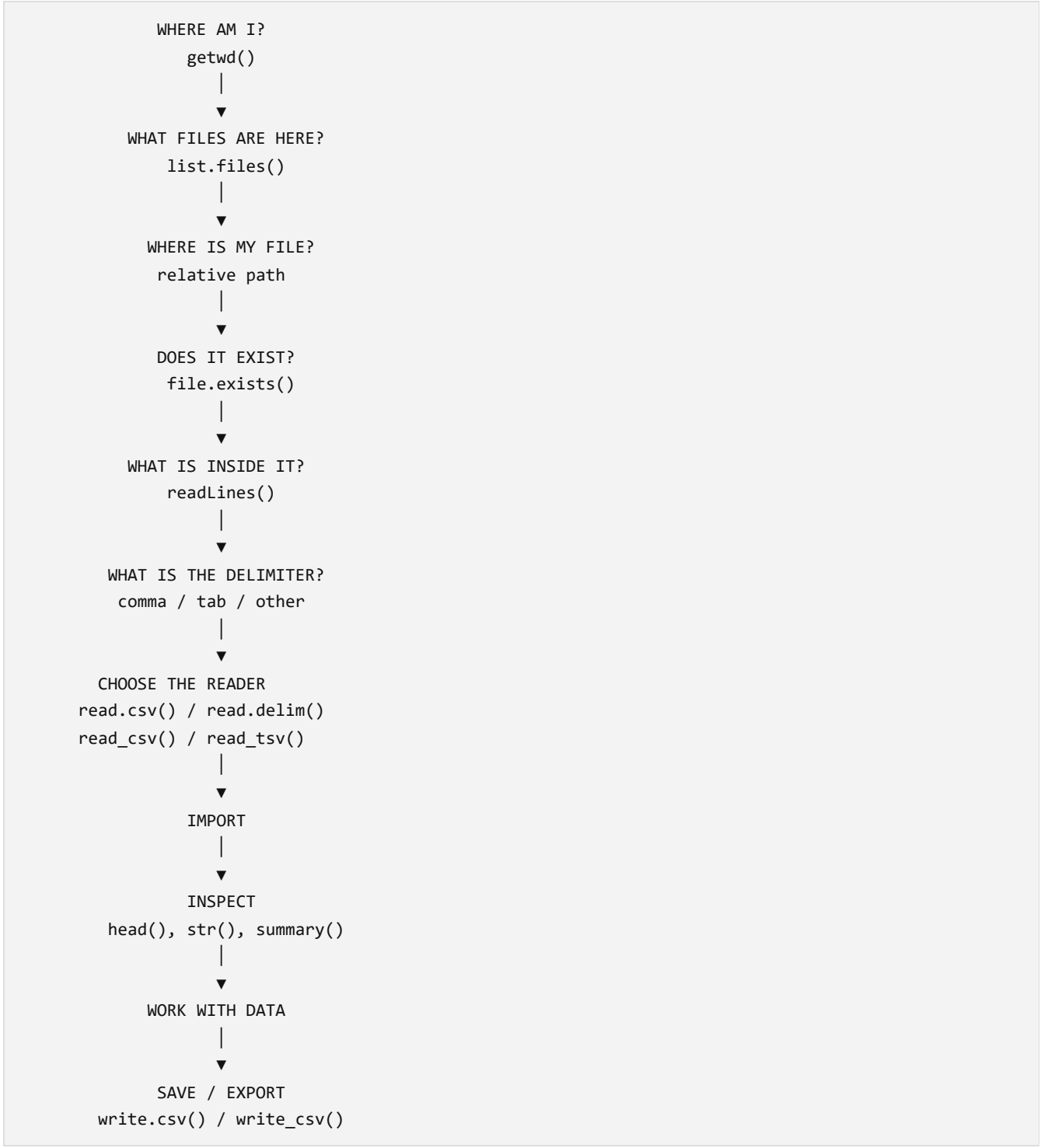
```
library(tidyverse)

# Import
students <- read_csv(
  "data/students.csv"
)

# Inspect
students
glimpse(students)
summary(students)

# Export
write_csv(
  students,
  "output/students_updated.csv"
)
```

Load `tidyverse` as in note **1.2**. We will learn additional tidyverse tools for manipulating data later.

---

28. The File Workflow to Remember

When working with an unfamiliar data file, ask these questions:



Three ideas to remember

> **1. A path tells R where a file is.**
>
> **2. A delimiter tells R how values inside a text file are separated.**
>
> **3. An import function reads the file and creates an R object that we can work with.**

For an unfamiliar text data file:

```
# Find/check it
file.exists("data/students.csv")

# Look inside
readLines("data/students.csv", n = 3)

# Choose the appropriate reader
students <- read.csv("data/students.csv")

# Check that it worked
head(students)
str(students)
```

This habit—**find** → **inspect** → **import** → **verify**—will prevent many common file-import problems.

Lab 1 asks you to import `students.csv`. Put it where a relative path can see it.